Jnana Vaahini Shikshana Samsthe, Mysore ®
Mysore College of Engineering and Management
(Approved by AICTE & Affiliated to VTU, Belagavi)
1072,T N Pura Road, Near Big Banyan Tree, Chikkhalli, Mysore-570028



Department of Computer Science and Engineering

INTERNET OF THINGS LABORATORY MANUAL

For Seventh Semester B.E - 2022 Batch [VTU/CBCS, 2022 syllabus]

Subject Code – BCS701

NAME:……..

USN :……

SECTION:……..

BATCH:…….

Presented by
Dr. Arunabh Kumar
Associate Professor
Department CSE

SYALLABUS

SEMESTER – VII

IOT LABORATORY

(Effective from the academic year 2026-2027)

Course Code – BCS701	**CIE Marks - 50**
Number of Contact Hours/Week -3:0:2:0	**SEE Marks - 50**
Total Number of Lab Contact Hours - 20	**Exam Hours - 03**

***COURSE LEARNING OBJECTIVES*:** This course will enable students to
1.Understand about the fundamentals of Internet of Things and its building blocks along with their characteristics.
2.Understand the recent application domains of IoT in everyday life.
3.Understand the protocols and standards designed for IoT and the current research on it.
4.Understand the other associated technologies like cloud and fog computing in the domain
5.Improve their knowledge about the various cutting-edge technologies in the field IoT and machine learning applications.
6.Gain insights about the current trends of machine learning and AI techniques used in IoT to orient towards the present industrial scenario.

Programs List:

Sl. No	**Experiments**
1.	Develop a program to blink 5 LEDs back and forth.
2.	Develop a program to interface a relay with Arduino board.
3.	Develop a program to deploy an intrusion detection system using Ultrasonic and sound sensors.
4.	Develop a program to control a DC motor with Arduino board.
5.	Develop a program to send and RX MSG using Bluetooth and Arduino board.
6.	Develop a program to classify dry and wet waste with the Moisture sensor (DHT22).
7.	Develop a program to detect the gas leakage in the surrounding environment.
8.	Develop a program to demonstrate weather station readings using Arduino.
9.	Develop a water level depth detection system using Ultrasonic sensor.
10.	Code to blink LED using with web server and Nodemcu ESP8266

COs	COURSE OUTCOMES
1	Explain the evolution of IoT, IoT networking components, and addressing strategies in IoT.
2	Analyze various sensing devices and actuator types.
3	Demonstrate the processing in IoT.
4	Apply different connectivity technologies.
5	Elaborate the need for Data Analytics and Security in IoT.

Laboratory Outcomes: The student should be able to:

- Develop and test programs using Arduino Uno and IoT Kit

  Conduct the experiments on an **Arduino Uno-based IoT development board** using **Arduino IDE** and required **sensor modules and communication components**.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The laboratory test **(duration 02 hours)** after completion of all the experiments shall be conducted for 10 marks**.**
- Write-up, evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

INTERNET OF THINGS LABORATORY

Introduction

The Internet of Things (IoT) is a revolutionary technology that connects the physical world to the digital world through a network of embedded devices, sensors, actuators, and cloud platforms. It enables the monitoring, automation, and intelligent decision-making of real-time systems across various domains such as healthcare, agriculture, smart cities, industrial automation, and environmental monitoring.

This laboratory course is designed to provide students with hands-on experience in building and testing basic to intermediate-level IoT applications using the Arduino Uno platform and associated sensor/actuator modules. Students will gain practical knowledge in sensor interfacing, device control, communication protocols.

Through a series of real-world focused experiments, learners will explore how to:

- Control and automate physical devices (e.g., LEDs, DC motors, relays),
- Monitor environmental parameters (e.g., temperature, gas, moisture, pH),
- Detect intrusions and water levels,
- Interface with cloud platforms for data visualization,
- Work with input devices like keypads,
- Understand and implement communication protocols.
- The experiments included in this manual cover a wide range of IoT applications such as:
- Smart lighting systems using LDRs,
- Intrusion detection using ultrasonic and sound sensors,
- Weather stations and environmental monitoring,
- Smart waste classification using DHT sensors,
- Gas leak detection and water level monitoring,
- UART communication and keypad interfacing.

ESP32 TRAINER KIT



1. **ESP32 Microcontroller (Top-left):**
   The heart of the board, ESP32 is a powerful microcontroller with built-in Wi-Fi and Bluetooth, ideal for IoT applications.

2. **Pre-connected Sensors and Modules:**
   Includes commonly used IoT sensors like Ultrasonic sensor (HC-SR04), Soil Moisture Sensor, DHT11/DHT22 (Temperature & Humidity), LDR (Light sensor), Pressure sensor, and Gas sensor for real-time environmental monitoring.

3. **Relay Module (Bottom-right):**
   Used to control high voltage devices like bulbs, fans, etc., making it suitable for home automation projects.

4. **OLED Display (Middle-top):**
   A 128x64 OLED screen is available for real-time data visualization like sensor values or status messages.

5. **Built-in LEDs, Switches & Buzzer (Bottom-left):**
   For basic input-output testing: LEDs for output indication, push buttons as digital inputs, and buzzer for sound alerts.

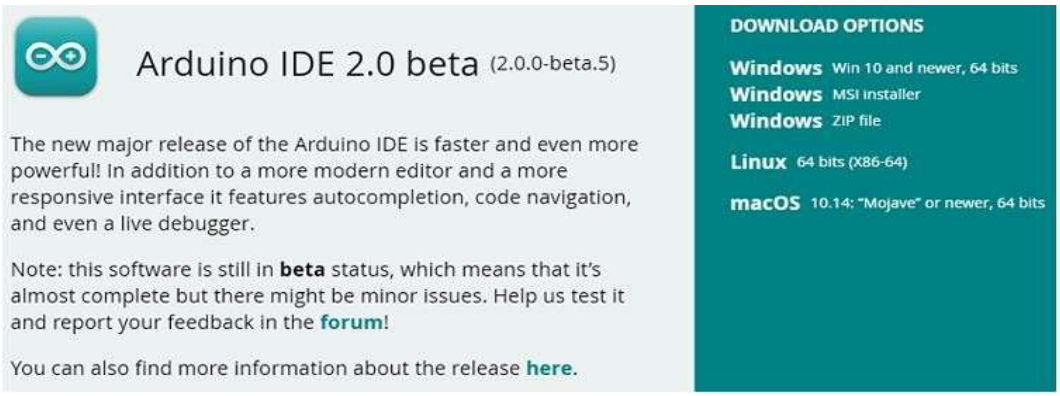
Arduino IDE 2.0 Installation

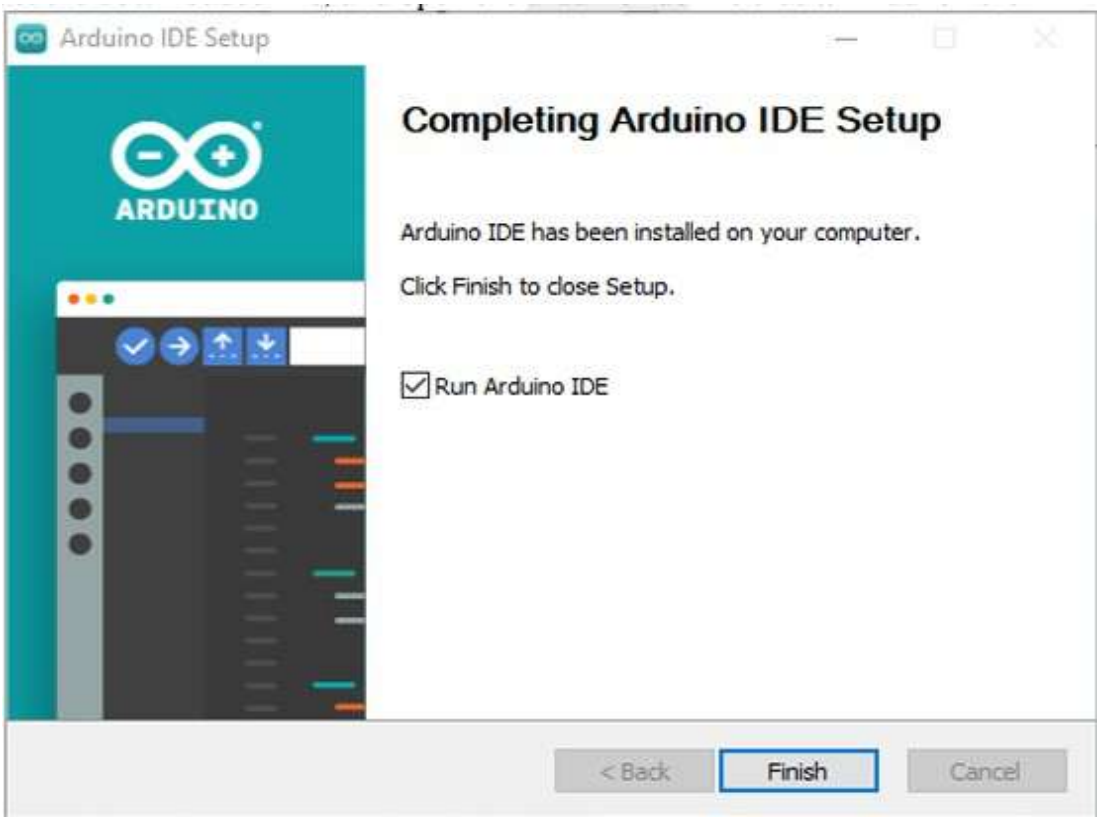
Before proceeding make sure you have Arduino IDE 2.0 installed in your computer. Go to the Arduino website and download the version for your operating system.

▪ Windows: run the file downloaded and follow the instructions in the installation guide.

▪ Mac OS X: copy the downloaded file into your application folder.

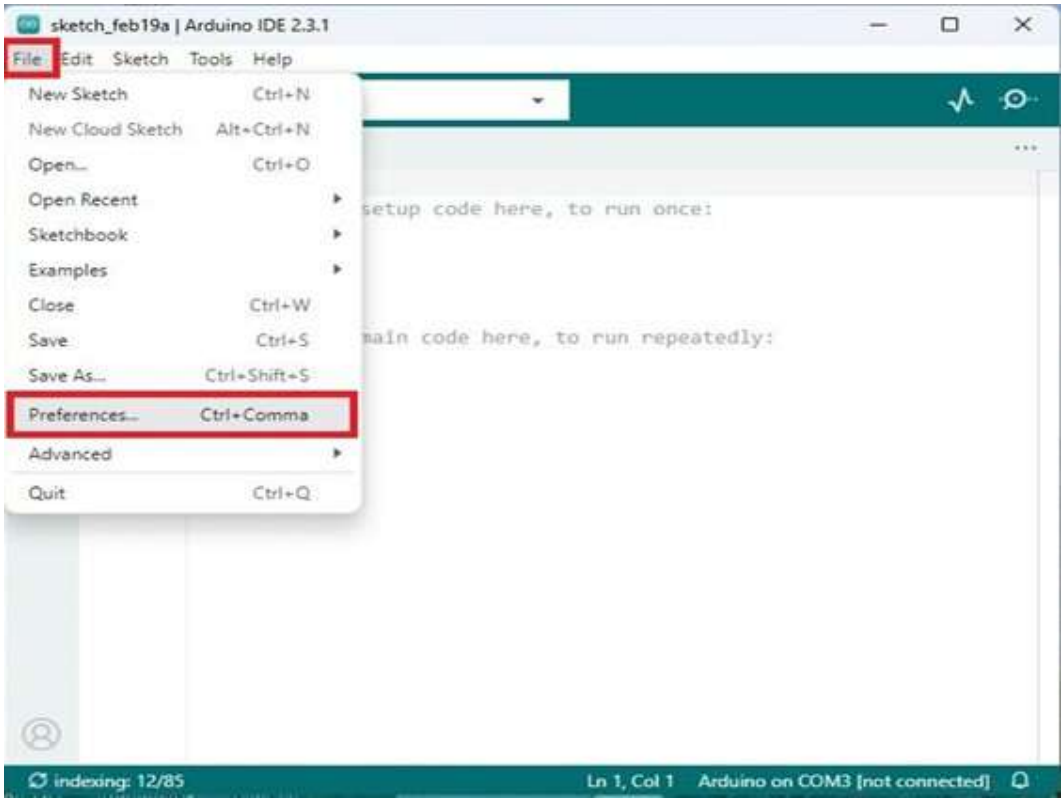
▪ Linux: extract the downloaded file, and open the arduino-ide file that will launch the IDE.



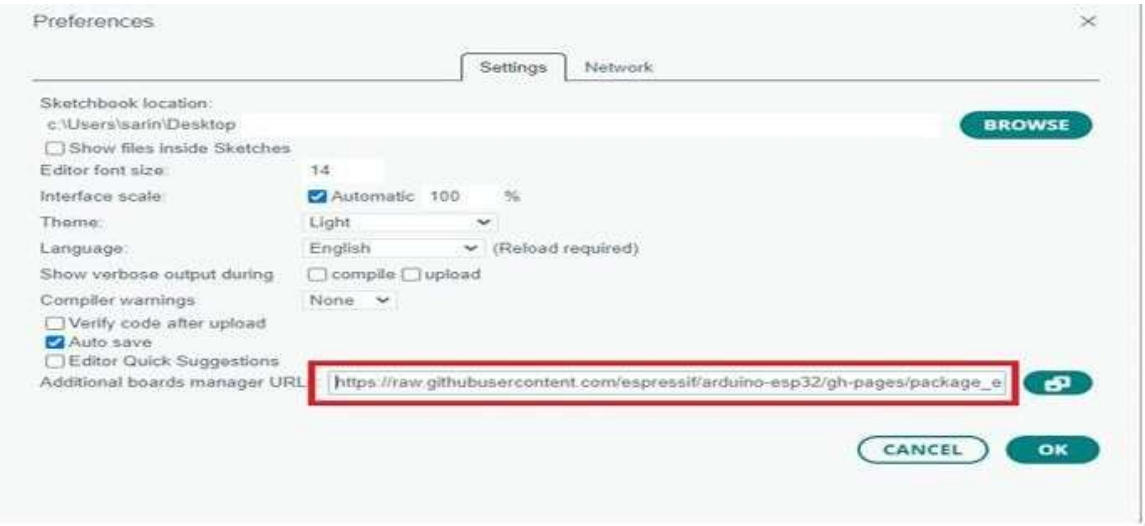


If you have doubts, you can go to the Arduino Installation Guide. To install the ESP32 board in your Arduino IDE, follow these next instructions:

1. In your Arduino IDE 2.0, go to File > Preferences.

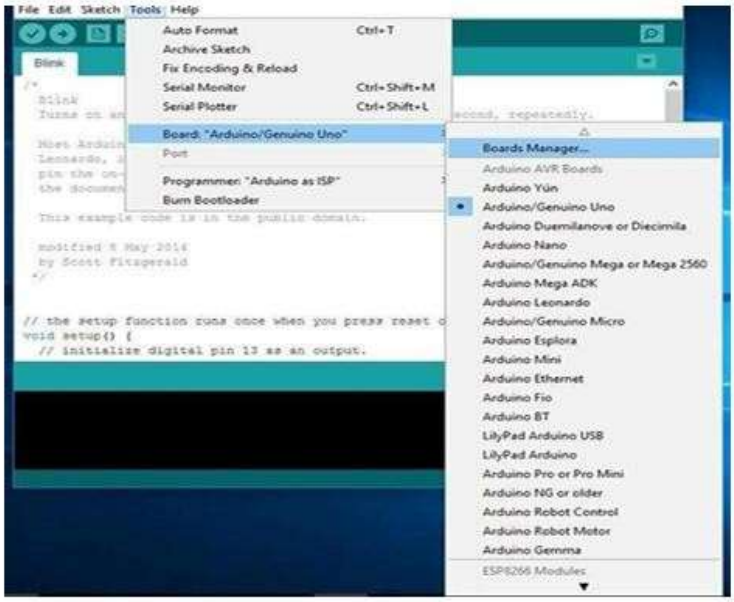


2. Copy and paste the following line to the Additional Boards Manager URLs field. http://arduino.esp8266.com/stable/package_esp8266com_index.json,https://dl.espressif.com/dl/package_esp32_index.json Adding ESP32 Board Manager



In the Additional Boards Manager enter below URL.
http://arduino.esp8266.com/stable/package_esp8266com_index.json,https://dl.espressif.com/dl/package_esp32_index.jsonAs highlighted in the figure and enter OK.

Selecting Board



Now open the tools in that select **Board: "Arduino/Genuino Uno"** and click on the **Boards Manager** as shown in the figure

ESP32 Board Package

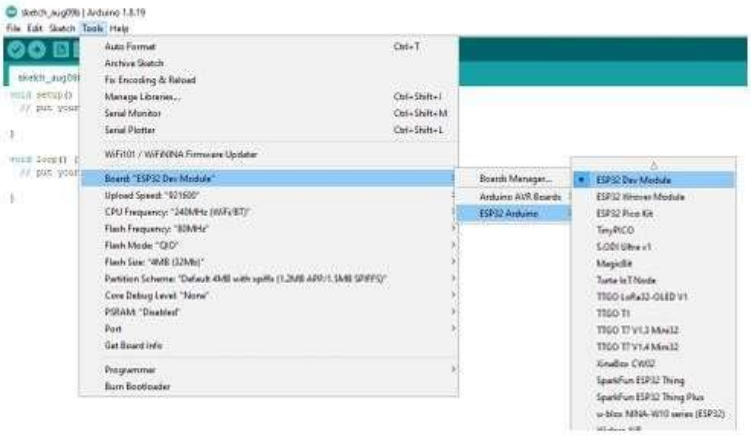


The Boards Manager window opens, scroll the window page to bottom till you see the module with the name ESP32.

Once we get it, select that module and select version and click on the Install button.

When it is installed it shows Installed in the module as shown in the figure and then closes the window.

Selecting ESP32 Arduino Board



To run the esp8266 with Arduino we have to select the **Board: "Arduino/Genuino Uno"** and then change it to "**ESP32 Dev Module"** or other esp8266 modules depending on what you have to select and select port, Upload the program.

Lab Manual for CSE IOT Lab

1. Develop a program to blink 5 LEDs back and forth. Using Arduino.

Code:-

```
// Define LED pins
const int ledPins[] = {2, 3, 4, 5, 6}; // 5 LEDs
const int ledCount = 5;

void setup() {
  // Set all LED pins as OUTPUT
  for (int i = 0; i < ledCount; i++) {
    pinMode(ledPins[i], OUTPUT);
  }
}

void loop() {
  // Move forward
  for (int i = 0; i < ledCount; i++) {
    lightOneLed(i);
    delay(200); // Delay between blinks
  }

  // Move backward
  for (int i = ledCount - 2; i > 0; i--) { // Avoid repeating ends
    lightOneLed(i);
    delay(200);
  }
}

void lightOneLed(int index) {
  // Turn off all LEDs first
  for (int i = 0; i < ledCount; i++) {
    digitalWrite(ledPins[i], LOW);
```

```
  }

  // Turn on the selected LED

  digitalWrite(ledPins[index], HIGH);

}
```

2. Develop a program to interface a relay with Arduino board.

Code:-

```
// Define the relay control pin
const int relayPin = 7;

void setup() {
  // Initialize relay pin as output
  pinMode(relayPin, OUTPUT);

  // Initially turn the relay off
  digitalWrite(relayPin, LOW); // or HIGH, depending on your relay logic
}
void loop() {
  // Turn relay ON
  digitalWrite(relayPin, HIGH);  // Change to LOW if your relay is active-low
  delay(1000); // Wait for 1 second

  // Turn relay OFF
  digitalWrite(relayPin, LOW);   // Change to HIGH if your relay is active-low
  delay(1000); // Wait for 1 second
}
```

3. Develop a program to deploy an intrusion detection system using Ultrasonic and sound sensors

Code:-

```
// Ultrasonic sensor pins
const int trigPin = 9;
const int echoPin = 10;

// Sound sensor pin (analog or digital)
const int soundPin = A0;  // use D2 if digital version

// Alert output (buzzer or LED)
const int alertPin = 8;

// Detection thresholds
const int distanceThreshold = 50; // in cm
const int soundThreshold = 500;   // analog sound level (0-1023)

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(alertPin, OUTPUT);

  Serial.begin(9600);
}
void loop() {
  // Measure distance
  long duration;
  int distance;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
```

```
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.034 / 2; // convert to cm

  // Read sound level
  int soundLevel = analogRead(soundPin);

  // Debug output
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.print(" cm | Sound: ");
  Serial.println(soundLevel);
  // Check for intrusion
  if (distance < distanceThreshold || soundLevel > soundThreshold) {
    digitalWrite(alertPin, HIGH); // Trigger alert
  } else {
    digitalWrite(alertPin, LOW);  // No intrusion
  }

  delay(200); // short delay between checks
}
```

4. Develop a program to control a DC motor with Arduino board

Code:-

```
// Motor control pins
const int in1 = 8;
const int in2 = 9;
const int enA = 10; // PWM pin for speed control
void setup() {
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(enA, OUTPUT);
}
void loop() {
  // Motor forward
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  analogWrite(enA, 200); // Speed (0-255)
  delay(2000); // Run forward for 2 seconds
  // Stop
  analogWrite(enA, 0);
  delay(1000);
  // Motor backward
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  analogWrite(enA, 150); // Slower speed
  delay(2000);
  // Stop
  analogWrite(enA, 0);
  delay(1000);
}
```

5. Develop a program to code to send and RX MSG using Bluetooth device and Arduino board.

Code:-

```
#include <SoftwareSerial.h>

// HC-05
// Arduino RX = 10
// Arduino TX = 11
SoftwareSerial Bluetooth(10, 11);

void setup() {
  Serial.begin(9600);
  Bluetooth.begin(9600);
```

```
  Serial.println("Bluetooth Ready");
  Serial.println("Type a message and press Enter...");
}

void loop() {

  // Send message from Serial Monitor to Bluetooth
  if (Serial.available()) {

    String message = Serial.readStringUntil('\n');

    Bluetooth.println(message);

    Serial.print("TX: ");
    Serial.println(message);
  }

  // Receive message from Bluetooth
  if (Bluetooth.available()) {

    String message = Bluetooth.readStringUntil('\n');

    Serial.print("RX: ");
    Serial.println(message);
  }
              }
```

6. Develop a program to classify dry and wet waste with the Moisture sensor (DHT22).

Code:-

```
#include "DHT.h"
#define DHTPIN 2         // DHT data pin
#define DHTTYPE DHT22    // Using DHT22
DHT dht(DHTPIN, DHTTYPE);
const int ledPin = 8;    // LED or buzzer to indicate wet waste
const float humidityThreshold = 60.0;  // Threshold to classify as wet
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(ledPin, OUTPUT);
}
void loop() {
  float humidity = dht.readHumidity();
  // Check if reading is valid
  if (isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
  if (humidity >= humidityThreshold) {
    Serial.println("Wet Waste Detected");
    digitalWrite(ledPin, HIGH); // Turn on indicator
  } else {
    Serial.println("Dry Waste Detected");
    digitalWrite(ledPin, LOW); // Turn off indicator
  }
  delay(2000); // Wait 2 seconds before next reading
}
```

7. Develop a program to detect the gas leakage in the surrounding environment.

Code:-

```
const int gasSensorPin = A0; // Analog output from MQ sensor
const int alarmPin = 8;      // Buzzer or LED
const int gasThreshold = 400; // Adjust based on testing
```

```
void setup() {
  pinMode(alarmPin, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int gasLevel = analogRead(gasSensorPin);
  Serial.print("Gas Level: ");
  Serial.println(gasLevel);
  if (gasLevel > gasThreshold) {
    Serial.println("⚠️ Gas Leakage Detected!");
    digitalWrite(alarmPin, HIGH); // Turn on alarm
  } else {
    digitalWrite(alarmPin, LOW);  // Safe
  }
  delay(1000);
}
```

8. Develop a program to demonstrate weather station readings using Arduino.

Code:-

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP085.h> // Use Adafruit_BMP280.h if using BMP280
#include <DHT.h>
// DHT Sensor Setup
#define DHTPIN 2         // DHT22 data pin connected to D2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
// BMP Sensor Setup
Adafruit_BMP085 bmp; // Change to Adafruit_BMP280 bmp; if using BMP280
void setup() {
  Serial.begin(9600);
  dht.begin();

  if (!bmp.begin()) {
    Serial.println("BMP sensor not detected!");
    while (1); // Halt
  }
}
void loop() {
  // Read from DHT22
  float temperature = dht.readTemperature(); // Celsius
  float humidity = dht.readHumidity();
  // Read from BMP180
  float pressure = bmp.readPressure() / 100.0; // hPa
  float altitude = bmp.readAltitude(1013.25);  // Adjust sea level pressure as needed
  // Display on Serial Monitor
  Serial.println("------ Weather Station ------");
  Serial.print("Temperature: "); Serial.print(temperature); Serial.println(" °C");
  Serial.print("Humidity   : "); Serial.print(humidity); Serial.println(" %");
  Serial.print("Pressure   : "); Serial.print(pressure); Serial.println(" hPa");
  Serial.print("Altitude   : "); Serial.print(altitude); Serial.println(" m");
  Serial.println("-----------------------------\n");
  delay(2000); // Delay between readings
}
```

9. Develop a water level depth detection system using Ultrasonic sensor.

Code:-

```
// Define pins
const int trigPin = 9;
const int echoPin = 10;
```

```
// Tank parameters
const float tankHeight = 100.0; // in cm (adjust based on your tank)

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  // Send ultrasonic pulse
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read echo time
  long duration = pulseIn(echoPin, HIGH);

  // Convert to distance in cm
  float distance = duration * 0.034 / 2;

  // Calculate water level
  float waterLevel = tankHeight - distance;

  // Print readings
  Serial.print("Distance from Sensor: ");
  Serial.print(distance);
  Serial.println(" cm");

  Serial.print("Water Level: ");
  Serial.print(waterLevel);
  Serial.println(" cm");

  // Optional: Water level alerts
  if (waterLevel < 20) {
    Serial.println("⚠️ Low water level!");
  } else if (waterLevel > 90) {
    Serial.println("✅ Tank is almost full.");
  }
  Serial.println("----------------------------");
  delay(2000);
}
```

10. Code to blink LED using with web server and Nodemcu ESP8266.

```
#include <ESP8266WiFi.h>
// Wi-Fi credentials
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASSWORD";

// LED pin
const int ledPin = D1; // GPIO5
bool ledState = false;

// WiFi server on port 80
WiFiServer server(80);

void setup() {
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  Serial.begin(115200);
  delay(100);

  // Connect to Wi-Fi
  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nConnected to WiFi.");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    Serial.println("New Client Connected");

    String request = client.readStringUntil('\r');
    Serial.println(request);
    client.flush();

    // LED control based on URL
    if (request.indexOf("/LED=ON") != -1) {
      digitalWrite(ledPin, HIGH);
      ledState = true;
    } else if (request.indexOf("/LED=OFF") != -1) {
      digitalWrite(ledPin, LOW);
      ledState = false;
    }

    // HTML response
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: text/html");
    client.println();

    client.println("<!DOCTYPE HTML>");
    client.println("<html>");
    client.println("<head><title>ESP8266 LED Control</title></head>");
    client.println("<body style='text-align:center;'>");
    client.println("<h1>LED is currently: " + String(ledState ? "ON" : "OFF") + "</h1>");
    client.println("<a href=\"/LED=ON\"><button style='font-size:20px;'>Turn ON</button></a>");
    client.println("<a href=\"/LED=OFF\"><button style='font-size:20px;'>Turn OFF</button></a>");
    client.println("</body></html>");

    delay(1);
    client.stop();
    Serial.println("Client Disconnected");
  }
}
```